```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv('/content/viral_videos.csv')
display(df.head())
```

	category	duration_sec	title_length	thumbnail_face	upload_hour	upload_day	tags_
0	Beauty	517	64	1	23	Wed	
1	Tech	495	47	1	23	Mon	
2	Beauty	311	76	0	17	Mon	
3	Fitness	590	69	0	0	Mon	
4	Tech	380	58	1	19	Mon	

```python
import pandas as pd
from sklearn.model_selection import train_test_split

# Identify features (X) and target (y)
y = df['viral']

# Exclude post-publication metrics to avoid data leakage
leakage_features = ['views', 'likes', 'comments', 'shares']
X = df.drop(columns=['viral'] + leakage_features)

# One-hot encode categorical features if any exist
X = pd.get_dummies(X, drop_first=True)

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_st

print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (9600, 27)
Testing set shape: (2400, 27)
```

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Initialize and train the Random Forest Classifier on non-leakage features
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predictions
y_pred = rf_model.predict(X_test)

```

```
# Evaluation Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

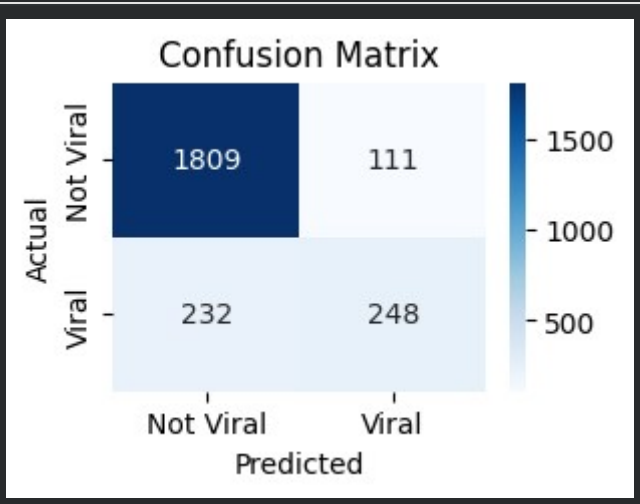
```
Accuracy: 0.8570833333333333

Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.94      0.91      1920
           1       0.69      0.52      0.59       480

    accuracy                           0.86      2400
   macro avg       0.79      0.73      0.75      2400
weighted avg       0.85      0.86      0.85      2400
```

```
# Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(3, 2))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Viral',
'Viral'], yticklabels=['Not Viral', 'Viral'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```




```
import numpy as np

# Get feature importances from the trained model
importances = rf_model.feature_importances_
feature_names = X.columns

# Create a DataFrame for visualization
feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

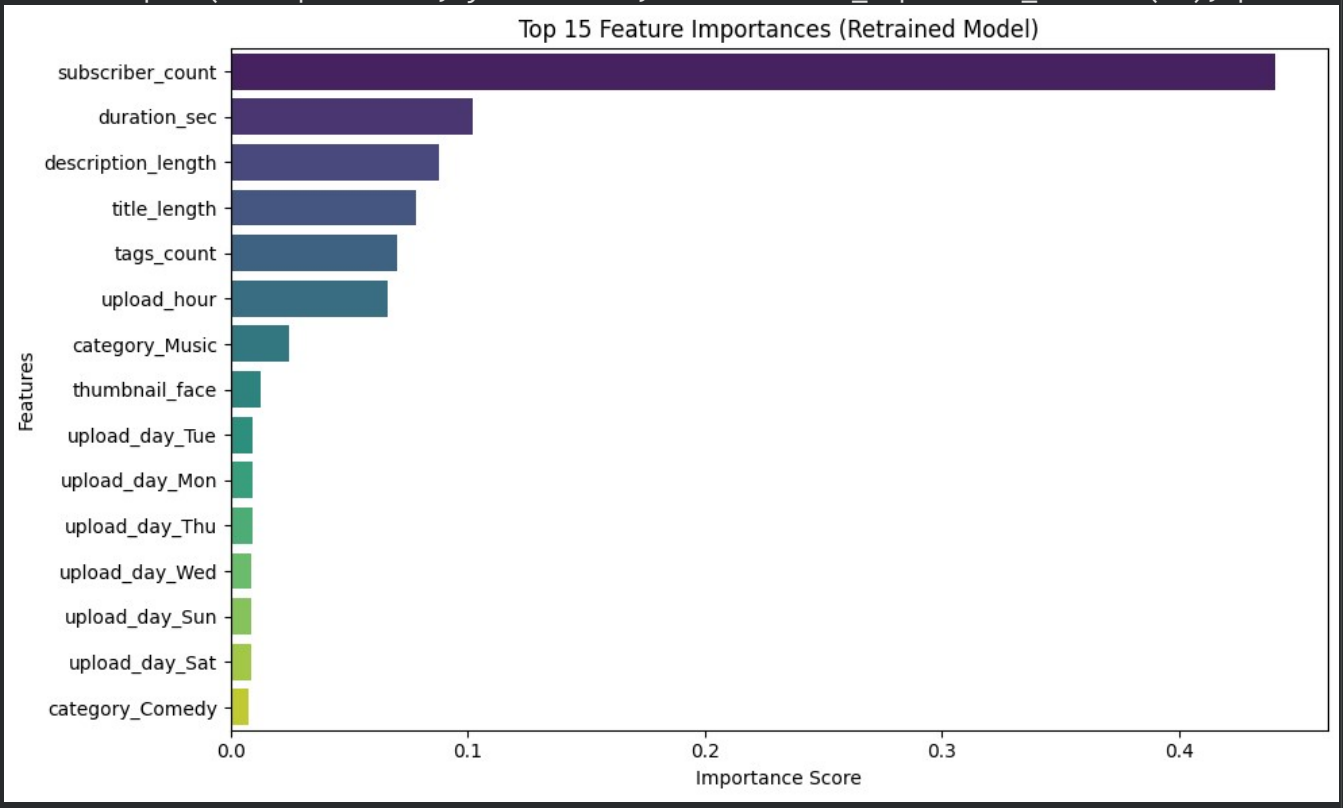
# Plot Feature Importances (Top 15 for readability)
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=feature_importance_df.head(15), pale
plt.title('Top 15 Feature Importances (Retrained Model)')
plt.xlabel('Importance Score')
plt.ylabel('Features')
```

```
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_4015/645102012.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.barplot(x='Importance', y='Feature', data=feature_importance_df.head(15), palette
```



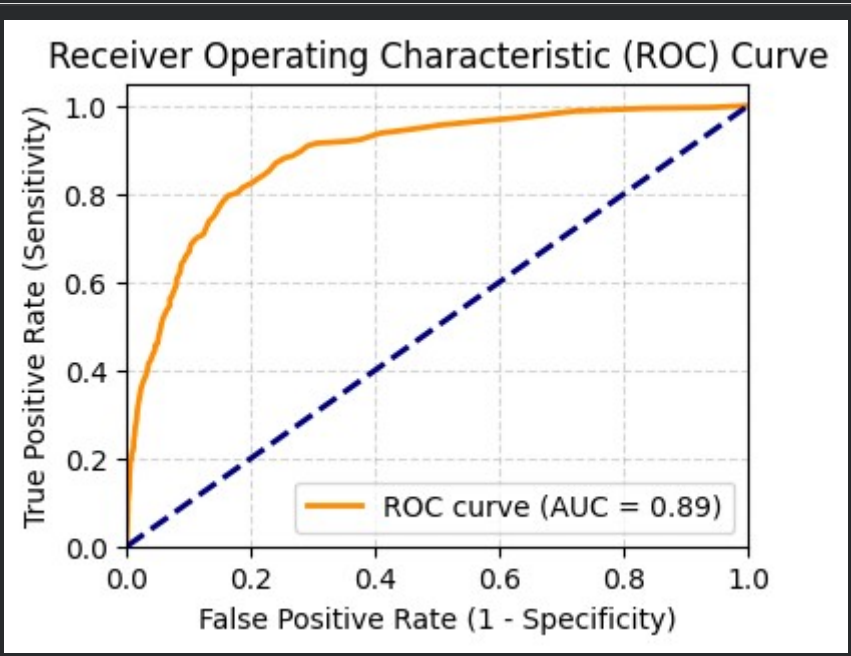

```
from sklearn.metrics import roc_curve, roc_auc_score

# Get predicted probabilities for the positive class (viral)
y_pred_prob = rf_model.predict_proba(X_test)[:, 1]

# Calculate ROC curve and ROC AUC score
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
auc_score = roc_auc_score(y_test, y_pred_prob)

# Plot ROC Curve
plt.figure(figsize=(4, 3))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC =
{auc_score:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)')
```

```
plt.ylabel('True Positive Rate (Sensitivity)')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```




```
import ipywidgets as widgets
from IPython.display import display, HTML

# Create interactive input widgets
subscriber_count = widgets.IntSlider(value=500000, min=0, max=10000000, step=10000, description='Subscribers:')
duration_sec = widgets.IntSlider(value=600, min=1, max=3600, step=10, description='Duration (s):')
description_length = widgets.IntSlider(value=500, min=0, max=5000, step=50, description='Desc. Length:')
title_length = widgets.IntSlider(value=50, min=1, max=100, step=1, description='Title Length:')
tags_count = widgets.IntSlider(value=15, min=0, max=50, step=1, description='Tags Count:')
upload_hour = widgets.IntSlider(value=12, min=0, max=23, step=1, description='Upload Hour:')
thumbnail_face = widgets.Dropdown(options=[('Yes', 1), ('No', 0)], value=1, description='Face in Thmb:')

categories = [col.replace('category_', '') for col in X.columns if col.startswith('category_')]
category_dropdown = widgets.Dropdown(options=categories, value=categories[0] if categories else None, description='Category:')

days = [col.replace('upload_day_', '') for col in X.columns if col.startswith('upload_day_')]
day_dropdown = widgets.Dropdown(options=days, value=days[0] if days else None, description='Upload Day:')

predict_btn = widgets.Button(description="Predict Virality", button_style='primary')
output = widgets.Output()

def make_prediction(b):
    with output:
```

```
        output.clear_output()

        # Create a single row template matching training features
        input_row = pd.DataFrame(0, index=[0], columns=X_train.columns)

        # Set numerical features
        input_row['subscriber_count'] = subscriber_count.value
        input_row['duration_sec'] = duration_sec.value
        input_row['description_length'] = description_length.value
        input_row['title_length'] = title_length.value
        input_row['tags_count'] = tags_count.value
        input_row['upload_hour'] = upload_hour.value
        input_row['thumbnail_face'] = thumbnail_face.value

        # Set categorical features
        cat_col = f'category_{category_dropdown.value}'
        if cat_col in input_row.columns:
            input_row[cat_col] = 1

        day_col = f'upload_day_{day_dropdown.value}'
        if day_col in input_row.columns:
            input_row[day_col] = 1

        # Get probability using our trained model
        prob = rf_model.predict_proba(input_row)[:, 1][0]
        pred = "VIRAL" if prob >= 0.5 else "NOT VIRAL"

        color = "green" if pred == "VIRAL" else "red"
        display(HTML(f"<h3>Prediction: <span style='color:{color};'>{pred}</span></h3>"))
        display(HTML(f"<p>Estimated Probability of Virality: <b>{prob*100:.1f}%</b></p>"))

predict_btn.on_click(make_prediction)

# Group and display layout
left_box = widgets.VBox([subscriber_count, duration_sec, description_length, title_length, tags_count])
right_box = widgets.VBox([upload_hour, thumbnail_face, category_dropdown, day_dropdown, predict_btn])
ui = widgets.HBox([left_box, right_box])

display(ui, output)
```

Subscribers: 500000

Duration (s): 600

Desc. Leng… 500

Title Length: 50

Tags Count: 15

Upload Hour: 12

Face in Th… Yes

Category: Comedy

Upload Day: Mon

Predict Virality